

XMR — Advanced Medical Report Analyzer

Complete Project Documentation

Last Updated: April 13, 2026

Table of Contents

1. Executive Summary
 2. Problem Statement
 3. Solution Overview
 4. Key Features
 5. Technology Stack
 6. System Architecture
 7. AI & Machine Learning Components
 8. Data Flow & Processing Pipeline
 9. Frontend Application
 10. Backend API
 11. Database Design
 12. Security & Privacy
 13. Third-Party Integrations
 14. Deployment & Infrastructure
 15. Unique Differentiators
 16. Demo Walkthrough
-

1. Executive Summary

XMR (Advanced Medical Report Analyzer) is a full-stack, AI-powered health management platform that transforms how patients understand and manage their medical data. The platform allows users to upload medical reports in any format — PDFs, scanned documents, or images — and instantly receive AI-analyzed, human-readable health insights.

Beyond simple analysis, XMR features a **real-time AI voice assistant** that users can have a live conversation with about their health reports, powered by WebRTC technology. All data is securely stored and visualized through interactive trend charts, enabling users to track their health over time.

Core Value Proposition: Bridge the gap between raw medical lab reports and actionable, personalized health understanding — making healthcare data accessible to everyone, not just medical professionals.

2. Problem Statement

Medical reports are filled with technical jargon, cryptic abbreviations, and numerical values with no intuitive context. Patients face several challenges:

- **Comprehension Gap:** Test names like “HbA1c,” “eGFR,” or “TSH” are meaningless without medical training.
- **Reference Ranges:** Values printed as “normal” may not be appropriate for a patient’s specific age, gender, or health context.
- **Fragmented History:** Results scattered across multiple paper documents, labs, and years make trend analysis impossible.
- **Access Barrier:** Patients cannot ask follow-up questions about their results without scheduling another appointment.
- **Format Diversity:** Lab reports come from dozens of different labs in different formats (PDF, image, scanned documents), making automated processing difficult.

3. Solution Overview

XMR addresses these problems through a multi-layered intelligent platform:

Challenge	XMR Solution
Incomprehensible reports	AI-powered natural language explanations per test
Inconsistent reference ranges	Age and gender-based dynamic range assignment
Fragmented health history	Centralized cloud database with trend visualization
No follow-up access	Real-time AI voice medical assistant (available 24/7)
Diverse report formats	Hybrid OCR pipeline (Gemini Vision + OpenCV preprocessing)

4. Key Features

4.1 AI-Powered Medical Report Analysis

- Accepts PDFs, images (PNG/JPG), and scanned documents
- Two-phase analysis: immediate regex-based results followed by comprehensive LLM analysis
- Extracts test names, values, units, reference ranges, and patient demographics automatically
- Groups tests by date when a report spans multiple visits

4.2 Real-Time AI Voice Medical Assistant

- Live voice conversation powered by LiveKit WebRTC infrastructure
- Professional-grade voice synthesis via Cartesia TTS/STT
- Context-aware — the assistant knows the contents of the user’s uploaded report
- Multilingual support with automatic language detection
- Voice activity detection (VAD) using Silero for natural conversation flow

4.3 Health Trend Tracking

- Interactive charts built with Chart.js visualizing test values over time
- Highlights abnormal trends across multiple reports
- Enables early detection of deteriorating health markers

4.4 Personalized Health Insights

- BMI calculation and classification
- Dietary and lifestyle recommendations based on test results
- Identification of concerning findings with plain-language explanations
- Age-appropriate interpretation of every result

4.5 Secure Report Management

- Upload, view, and delete historical reports
- Permanent cloud storage via Cloudinary
- Patient name, age, and gender extracted automatically from reports

4.6 Enterprise-Grade Security

- OTP-based email authentication
- JWT token authorization for all API calls
- Row Level Security (RLS) enforced at the database level
- Each user can only access their own medical data

5. Technology Stack

Backend

Technology	Purpose
Python 3.12	Core language
Flask	REST API framework
Google Gemini 2.5 Flash Lite (<code>google.gemai</code>)	Primary LLM — text extraction, analysis, explanations, vision/OCR

Technology	Purpose
OpenCV	Image preprocessing (6 enhancement techniques)
PyMuPDF (fitz)	PDF text layer extraction
pdfplumber	Fallback PDF parsing
Supabase (PostgreSQL)	Primary database
Cloudinary	File storage and CDN
LiveKit Agents SDK	Voice agent infrastructure
Cartesia	TTS/STT voice synthesis
Silero VAD	Voice activity detection
Gunicorn	Production WSGI server
Docker	Containerization

Frontend

Technology	Purpose
React 18	UI framework
TypeScript	Type-safe development
Vite	Fast build tooling
Tailwind CSS	Utility-first styling
Chart.js + react-chartjs-2	Interactive data visualization
LiveKit React SDK	Voice interface components
Supabase JS	Authentication and realtime
React Router v7	Client-side routing
Lucide React	Icon library

Infrastructure & Cloud

Service	Role
Supabase	PostgreSQL database + auth + RLS
Cloudinary	Medical file storage + CDN
LiveKit Cloud	WebRTC real-time voice infrastructure
Google AI (VERTEX_AI_API_KEY)	Gemini 2.5 Flash Lite for all LLM + Vision calls
Docker / Render	Backend deployment
Vercel / Netlify	Frontend deployment

6. System Architecture

FRONTEND (React + TypeScript)

Dashboard (Upload)	Reports Page	TrendCharts (Chart.js)	VoiceAgent (LiveKit)
-----------------------	-----------------	---------------------------	-------------------------

BACKEND (Flask / Python)

/upload	/reports	/trends
/save-report	/report/id	/stats
	/delete	/download

analyzer.py	database.py
pdf_reader	(Supabase)
ocr_reader	

AI / LLM Layer

google.genai Client
gemini-2.5-flash-
lite (text + vision)

Cloud Infrastructure

Supabase (PostgreSQL)
Cloudinary (Storage)
LiveKit Cloud (WebRTC)
Cartesia (Voice)

Module Breakdown

server.py (1,185 lines) — Main Flask API server. Hosts all 15+ REST endpoints. Orchestrates the upload flow, authentication, and report management.

modules/analyzer.py (1,280 lines) — The AI analysis engine. Uses a single google.genai client (gemini-2.5-flash-lite) for all LLM calls: structured

data extraction, test explanations, personalized health insights, and date extraction. Includes a `_parse_llm_json()` helper that robustly handles all LLM response formats.

modules/pdf_reader.py — Multi-method PDF text extraction. Detects whether a PDF has a text layer or is a scanned image, routes to the appropriate method, and calls `gemini-2.5-flash-lite` for page-by-page analysis.

modules/ocr_reader.py — Gemini Vision OCR. Sends raw image bytes to `gemini-2.5-flash-lite` via the `google.genai` multimodal API. Falls back to OpenCV preprocessing (6 techniques) if the raw image yields poor results.

modules/database.py — All Supabase database operations (CRUD for reports, test results, trends). Includes a local JSON fallback for offline development.

livekit_agent.py (250 lines) — The real-time voice medical assistant. Manages a LiveKit session with the user, using Cartesia for TTS/STT and Silero for VAD.

main.py — Thin orchestration layer that ties the PDF reader, OCR reader, and analyzer together.

7. AI & Machine Learning Components

7.1 Single LLM Provider: Google Gemini 2.5 Flash Lite

All AI calls go through one client, authenticated via `VERTEX_AI_API_KEY`:

```
from google import genai
from google.genai import types

client = genai.Client(api_key=os.getenv("VERTEX_AI_API_KEY"))
VERTEX_MODEL_NAME = "gemini-2.5-flash-lite"
```

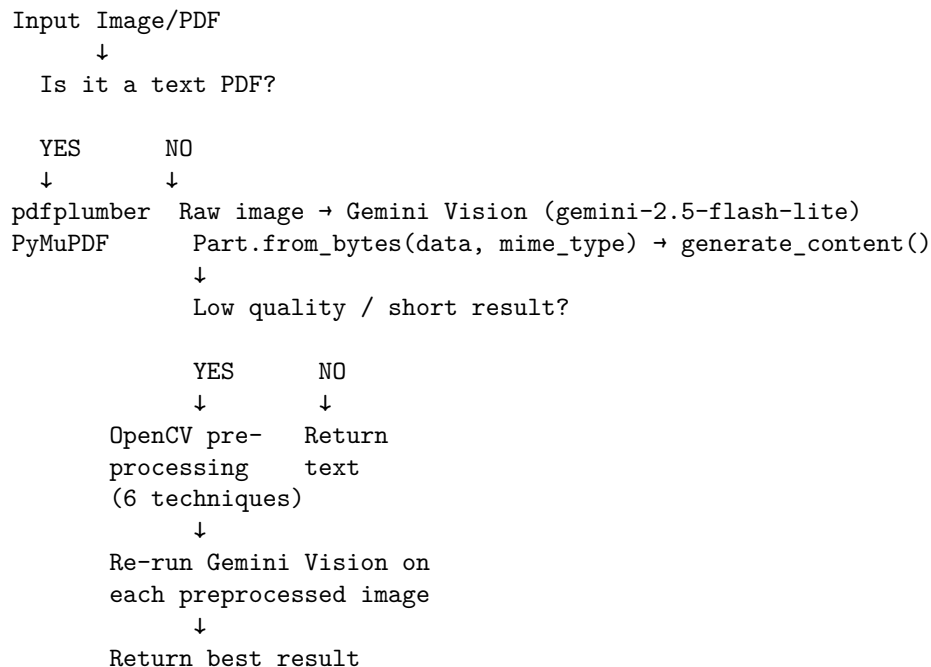
Why `gemini-2.5-flash-lite`? - Fastest latency in the Gemini 2.5 family - Supports multimodal input (text + images) — same model handles OCR and analysis - Strong JSON output formatting for structured medical data extraction - Cost-effective for high-volume report processing

7.2 LLM Use Cases

Call	Prompt Goal	Output
<code>extract_medical_data_gemini(text)</code>	Extract structured data from raw report text	JSON array with patient info, test values, dates
<code>generate_test_explanations_gemini(test_name)</code>	Explain test results in plain language	JSON object mapping test name → explanation

Call	Prompt Goal	Output
<code>generate_personalized_analysis()</code>	Medical analysis + dietary/lifestyle recs	JSON with summary, concerning findings, recommendations
<code>extract_dates_from_text(text)</code>	Extract all dates from report text	JSON array of YYYY-MM-DD date strings
<code>analyze_pdf_page_with_taglist()</code>	Picture extraction from PDFs	JSON per-page with date, patient, tests

7.3 Vision / OCR Pipeline



OpenCV preprocessing techniques (applied in sequence): 1. Bilateral filter + adaptive threshold 2. Gaussian blur + Otsu threshold 3. Median blur + CLAHE enhancement 4. Morphological operations (close + open) 5. High contrast enhancement 6. Upscaling for small fonts (< 1200px width)

7.4 Medical Knowledge Base

- Reference ranges for 20+ common tests: Hemoglobin, Glucose, Cholesterol, HbA1c, Creatinine, ALT, AST, and more
- Age-stratified ranges: pediatric, adult, and senior thresholds differ
- Gender-based adjustments (e.g., Hemoglobin ranges differ between male and female)

- Interpretation logic: values classified as **Low** / **Normal** / **High** / **Unknown**
- `_parse_llm_json()` helper: robustly strips markdown fences, handles truncated JSON, ensures the pipeline never crashes on malformed LLM output

7.5 LLM Response Format Handling

The LLM may return data in three different formats depending on the prompt. The pipeline normalises all of them:

LLM Response Format	Handler
<code>[{"patient_name": ..., "data_points": [...]}]</code> (array)	<code>convert_new_llm_format_to_legacy()</code>
<code>{"pages": [...]} (page-based)</code>	<code>convert_page_based_to_legacy()</code>
<code>{"patient": {...}, "tests": [...]} (legacy)</code>	Used directly

7.6 Voice AI (LiveKit + Cartesia)

- **Silero VAD**: Detects when the user starts/stops speaking
- **Cartesia STT**: Transcribes user speech with medical vocabulary awareness
- **LLM (Gemini)**: Processes the question in context of the uploaded report
- **Cartesia TTS**: Responds in natural, professional voice
- The assistant is initialized with the user's report contents so it can answer specific questions like "What does my Hemoglobin value of 12.0 mean for me?"

8. Data Flow & Processing Pipeline

File Upload Flow

1. User selects file on Dashboard
↓
2. Frontend sends multipart/form-data POST to `/upload`
(includes file + user profile: age, gender, name)
↓
3. `server.py` receives file, saves to temp directory
↓
4. `main.py:analyze_file()` called
↓
5. PHASE 1 - Fast Analysis (returns in ~2-5 seconds)
 - `pdf_reader.py` / `ocr_reader.py` extracts text
 - Regex pattern matching finds common test patterns

- Returns basic test list to frontend immediately
- ↓
- 6. PHASE 2 - Deep LLM Analysis (returns in ~10-20 seconds)
 - analyzer.py sends text to gemini-2.5-flash-lite
 - LLM returns structured JSON with all tests
 - Format normalised (list/page/legacy → standard dict)
 - Date grouping applied to multi-date reports
 - Reference ranges assigned per test
 - Interpretation determined (Low/Normal/High)
 - Personalized explanations + health summary generated
- ↓
- 7. File uploaded to Cloudinary for permanent storage
- ↓
- 8. Full results returned to frontend
- ↓
- 9. User can optionally save report to Supabase via /save-report
- ↓
- 10. Report persisted with all test_results records

Voice Conversation Flow

1. User clicks mic button on VoiceAgent component
- ↓
2. Frontend requests LiveKit JWT token from backend
- ↓
3. LiveKit session established (WebRTC)
- ↓
4. User speaks → Cartesia STT transcribes
- ↓
5. Silero VAD detects end of speech
- ↓
6. Transcript + report context sent to Gemini LLM
- ↓
7. LLM response → Cartesia TTS → audio stream
- ↓
8. User hears AI response in real time

9. Frontend Application

Page Structure

Route	Component	Description
/	LoginPage	Email/password login
/otp	OTPVerification	6-digit OTP code entry

Route	Component	Description
/onboarding	UserOnboarding	First-time profile setup (name, age, goals)
/dashboard	Dashboard	File upload + analysis results
/report	Report	Detailed test result view
/reports	ReportsPage	Historical report library
/report/:id	ReportDetail	Individual saved report
/trends	TrendChart	Health metric trend graphs

Key UI Components

Dashboard — The primary interface. Features drag-and-drop file upload, progressive loading (shows Phase 1 results while Phase 2 loads), and an embedded voice assistant panel.

Report — Renders test results in a color-coded table (red for High, blue for Low, green for Normal), with AI-generated explanations for each value. Groups results by date automatically.

VoiceAgent — Displays a visual waveform animation during recording. Shows conversation transcript. Handles LiveKit session lifecycle.

TrendChart — Renders selected test metrics on a time-series line chart. Users can switch between different tests to view progression over time.

Authentication Flow

```

LoginPage → Enter email + password
    ↓
POST /auth/login → Supabase validates credentials
    ↓
OTPVerification → User enters 6-digit code from email
    ↓
POST /auth/verify-otp → Server validates OTP (5-min expiry)
    ↓
AuthContext stores user state → Dashboard

```

10. Backend API

Authentication Endpoints

Method	Endpoint	Description
POST	/auth/login	Email/password login, triggers OTP send
POST	/auth/verify-otp	Verify 6-digit OTP code

Medical Report Endpoints

Method	Endpoint	Description
POST	/upload	Upload and analyze a medical report file
POST	/save-report	Save analyzed results to Supabase
GET	/reports/<user_id>	List all reports (paginated)
GET	/report/<report_id>	Fetch full report details
DELETE	/report/<report_id>	Delete a report and all results
POST	/download-txt	Export test results as plain text

Analytics Endpoints

Method	Endpoint	Description
GET	/trends/<user_id>/<test_name>	Historical values for a test
GET	/stats/<user_id>	Aggregate user health statistics

Utility Endpoints

Method	Endpoint	Description
POST	/extract-pdf-info	Google Vision PDF extraction
GET	/proxy-image	CORS proxy for Cloudinary images
GET	/test-db	Database connectivity check

11. Database Design

Schema Overview

users

```
id          UUID PRIMARY KEY DEFAULT gen_random_uuid()
email       TEXT UNIQUE NOT NULL
password_hash TEXT NOT NULL
created_at  TIMESTAMPTZ DEFAULT NOW()
-- RLS Policy: users can only SELECT/UPDATE their own row
```

medical_reports

```
id          UUID PRIMARY KEY DEFAULT gen_random_uuid()
user_id     UUID REFERENCES users(id) ON DELETE CASCADE
filename    TEXT
file_type   TEXT
```

```

file_url          TEXT          -- Cloudinary URL
extracted_tests   JSONB         -- Array of test names
patient_name      TEXT
patient_age       INTEGER
patient_gender    TEXT
created_at        TIMESTAMPTZ DEFAULT NOW()
-- RLS Policy: user_id must match authenticated user

test_results

id                UUID PRIMARY KEY DEFAULT gen_random_uuid()
report_id         UUID REFERENCES medical_reports(id) ON DELETE CASCADE
test_name         TEXT
value            TEXT
unit             TEXT
reference_range   TEXT
interpretation    TEXT          -- 'High' / 'Normal' / 'Low' / 'Unknown'
explanation        TEXT          -- AI-generated description
date_of_test      DATE          -- Extracted from report
created_at        TIMESTAMPTZ DEFAULT NOW()
-- Indexes: (report_id), (test_name), (interpretation), trends composite
-- RLS Policy: via report_id → user_id chain

otp_codes

id                UUID PRIMARY KEY DEFAULT gen_random_uuid()
user_id           UUID REFERENCES users(id)
email            TEXT
code             TEXT          -- 6-digit OTP
expires_at        TIMESTAMPTZ  -- NOW() + 5 minutes
verified          BOOLEAN DEFAULT FALSE
created_at        TIMESTAMPTZ DEFAULT NOW()

```

Row Level Security

All tables enforce RLS. Even if an API key is compromised, a user cannot read another user's medical data. The database itself is the last line of defense.

12. Security & Privacy

Authentication

- **OTP-based login** with 5-minute token expiry prevents credential stuffing
- **JWT tokens** sign all API requests and LiveKit sessions
- **Password hashing** using bcrypt before database storage

Data Protection

- **Row Level Security (RLS)** — Supabase enforces at the DB level that users only access their own records
- **CORS restrictions** — Backend only accepts requests from authorized frontend origins
- **Input validation** — File type and size validation before processing
- **No raw file storage on server** — Files are processed in temp directories and deleted, permanently stored on Cloudinary

Voice Sessions

- LiveKit JWT tokens are short-lived and scoped to a single room session
- Voice data is not retained after the session ends

Compliance Considerations

- Medical data is handled with sensitivity to HIPAA principles
 - User data is never shared with third parties beyond the listed service providers
 - Users can delete all their data at any time via the report delete function
-

13. Third-Party Integrations

Google Gemini 2.5 Flash Lite (`google.genai`)

- **What it does:** Every AI call in the platform — structured text extraction, vision/OCR, test explanations, personalized health analysis, date extraction, and PDF page analysis
- **Authentication:** `VERTEX_AI_API_KEY` (Google AI API key)
- **SDK:** `google-genai` (v1.72+) — the current, actively maintained Google AI Python SDK
- **Why:** Single model handles both text and multimodal (image) inputs, eliminating the need for separate OCR and LLM providers

Supabase

- **What it does:** PostgreSQL database, authentication infrastructure, and Row Level Security
- **Why:** Combines managed Postgres + auth + RLS in one service with excellent free tier

Cloudinary

- **What it does:** Stores all uploaded medical report files (PDFs and images)

- **Why:** Built-in CDN, transformations, and OCR capabilities; handles medical files reliably

LiveKit Cloud

- **What it does:** WebRTC infrastructure for real-time voice sessions between user and AI
- **Why:** Industry-leading low-latency audio; managed infrastructure with a generous SDK

Cartesia

- **What it does:** State-of-the-art speech-to-text and text-to-speech
- **Why:** Natural voice quality, multilingual support, and medical vocabulary recognition

14. Deployment & Infrastructure

Backend Deployment (Docker / Render)

```
FROM python:3.11-slim
# Installs: OpenCV dependencies, Poppler (for PDF rendering)
# Port: 10000
# WSGI: Gunicorn (2 workers, 300s timeout for long OCR operations)
```

Deployment targets: - **Render** — render.yaml configuration provided - **Railway** — Direct Python deployment compatible - **AWS / GCP** — Docker image deployable to any container service - **Heroku** — Buildpack compatible

Frontend Deployment

- npm run build generates static assets in dist/
- Deployable to **Vercel**, **Netlify**, or **GitHub Pages**
- Environment variables injected at build time (VITE_ prefix)

Environment Configuration

```
# AI - single key powers all LLM + Vision calls
VERTEX_AI_API_KEY=AQ.xxxx...      # Google AI API key for gemini-2.5-flash-lite
VERTEX_MODEL=gemini-2.5-flash-lite # Optional override

# Database
SUPABASE_URL=https://xxx.supabase.co
SUPABASE_ANON_KEY=eyJ...
SUPABASE_SERVICE_ROLE_KEY=eyJ...
```

```

# Storage
CLOUDINARY_URL=cloudinary://key:secret@cloud

# Voice
LIVEKIT_URL=wss://your-project.livekit.cloud
LIVEKIT_API_KEY=APIxxx
LIVEKIT_API_SECRET=xxx
CARTESIA_API_KEY=sk_car_xxx

# Server
PORT=5001

# Frontend (.env.local)
VITE_SUPABASE_URL=https://xxx.supabase.co
VITE_SUPABASE_ANON_KEY=eyJ...
VITE_BACKEND_URL=http://127.0.0.1:5001
VITE_LIVEKIT_URL=wss://your-project.livekit.cloud
VITE_LIVEKIT_API_KEY=APIxxx
VITE_LIVEKIT_API_SECRET=xxx
VITE_CARTESIA_API_KEY=sk_car_xxx

```

15. Unique Differentiators

1. One Model, Everything

Most platforms use separate services for OCR (e.g., Tesseract, Google Vision) and LLM analysis (e.g., OpenAI, Gemini). XMR uses a single model (`gemini-2.5-flash-lite`) for both — it reads the image directly and extracts structured medical data in one shot. Simpler architecture, fewer failure points, lower cost.

2. Two-Phase Analysis — Instant + Comprehensive

Users see results in ~3 seconds (Phase 1 regex analysis) while the comprehensive AI analysis loads in the background (~15 seconds). This eliminates perceived wait time without sacrificing accuracy.

3. Resilient JSON Parsing

The `_parse_llm_json()` helper handles all LLM response quirks: mark-down code fences, truncated output, extra whitespace. The LLM response format normalisation layer (`convert_new_llm_format_to_legacy`, `convert_page_based_to_legacy`) ensures the pipeline works regardless of which format the model returns.

4. Real-Time Voice AI with Medical Context

Unlike general voice assistants, XMR’s assistant is loaded with the user’s actual uploaded report. It can answer “Is my Hemoglobin of 12.0 concerning?” with direct reference to the specific report data — not generic information.

5. Date-Aware Report Parsing

Medical reports often include multiple tests from different dates. XMR’s AI automatically identifies and groups results by the date they were taken — preserving the clinical context of each set of results.

6. Personalization Layer

Results are interpreted in the context of the user’s profile: age, gender, body type, health goals, and medical history. A hemoglobin value that is “normal” for a 30-year-old man is not the same for a 70-year-old woman. XMR accounts for this.

16. Demo Walkthrough

Step 1: User Registration & Login

- User enters email and password on the login screen
- OTP is sent to their email; they enter the 6-digit code
- First-time users complete onboarding (name, age, health goals)

Step 2: Upload a Medical Report

- From the Dashboard, user clicks “Upload Report” or drags a PDF/image onto the upload area
- Phase 1 results appear within ~3 seconds showing basic test values
- Phase 2 detailed results populate over the next ~15 seconds

Step 3: Review AI Analysis

- Tests are displayed in a color-coded table (red = High, blue = Low, green = Normal)
- Each test has an AI-generated plain-language explanation personalized to the user’s profile
- Multi-date reports show results grouped by the date of each test
- Overall health summary, concerning findings, dietary and lifestyle recommendations shown

Step 4: Talk to the Voice Assistant

- User clicks the microphone button on the VoiceAgent panel

- “My Hemoglobin is 12.0 — should I be concerned?”
- Assistant responds with personalized, context-aware guidance in real time

Step 5: Save & Track Trends

- User saves the report to their account
- Navigates to the Trends page and selects “Hemoglobin” to see values charted over time
- Compares current results to past reports to understand health trajectory

Step 6: Report History

- All saved reports visible in the Reports Library
- Users can re-open any past report, view full details, or delete records

Summary

XMR is a production-ready, AI-native health platform that combines:

- **Single-model AI** — `gemini-2.5-flash-lite` via `google.genai` handles text extraction, vision OCR, explanations, and personalized analysis
- **Real-time conversational AI** — WebRTC + Cartesia + Gemini
- **Secure cloud infrastructure** — Supabase + Cloudinary + LiveKit
- **Personalized health analytics** — trend charts + age/gender-aware interpretation

The result is a platform that genuinely democratizes access to medical knowledge — giving every patient the ability to understand their health data as clearly as a doctor would explain it.

Document updated April 13, 2026 — reflects Gemini 2.5 Flash Lite migration via `google.genai` SDK